Birla Institute of Technology and Science, Pilani

Work Integrated Learning Programmes Division

Information Retrieval (AIMLCZG537 / DSECLZG537)

Assignment 2

Text Mining and Document Intelligence System for a Technology Knowledge Library

Technical Report - Group G25

Student Registration Number	Name
2025AG05042	Tilva Dhruv Bhaveshbhai
2025AG05041	Km Shweta Jain
2025AG05067	Pandya Swapnil Kaushik
2025AG05097	Shaik Jareena
2025AG05397	Mervana Jay Jitendrabhai

Date: 04 November 2026

1. Introduction

This report describes the Text Mining and Document Intelligence System built by Group G25 for Assignment 2. The system is a Streamlit web application. It loads a collection of documents, preprocesses the text, builds TF-IDF features and an inverted index, classifies documents with Naive Bayes, groups them with K-Means, and recommends related documents. TF-IDF, the inverted index, Naive Bayes, K-Means, Purity and Rand Index were all implemented by us in Python, using NumPy only for basic array arithmetic. No scikit-learn or other library implementation of these methods is used. A unit test checks this automatically.

All numbers in this report were produced by the code in the submission (`scripts/run_experiments.py`) with the default settings and random seed 42. The same values appear in the application under the same settings.

2. Selected Business Context

We assume that our group is the text mining team of a technology consulting firm. The firm keeps an internal **knowledge library** of reference articles that consultants read before client engagements, for example a fintech analytics project or an NLP proof of concept. The library covers five practice areas: Machine Learning, Language & Search Technology, AI Systems & Cognition, Computing & Infrastructure, and Finance & Economics.

The library is growing and is maintained by a small team. Analysts have three recurring needs:

- **Tagging:** every new article must be assigned to a practice area so it can be found (document classification).
- **Topic discovery:** knowledge managers want to see which themes actually occur in the collection and whether the existing practice areas still fit (document clustering).
- **Related reading:** a consultant reading one article wants the most related articles (document recommendation).

3. Problem Statement

Given a collection of unstructured, noisy text documents with category labels, build an interactive system that (1) ingests and explores the documents and detects duplicates, (2) preprocesses the text in a configurable way and represents documents with manually computed, length-normalised TF-IDF vectors and an inverted index, (3) classifies documents with a manually implemented Naive Bayes classifier, (4) clusters documents with a manually implemented K-Means algorithm evaluated by Purity and Rand Index, (5) recommends the Top-K related documents, and (6) visualises and compares the results.

4. Dataset Description

The dataset is the folder `Dataset/` with **73 plain-text files**, each an English Wikipedia article exported as text. Together they contain **491,271 words** (3.91 MB of text after the source footer is removed). The assignment asks for at least 50-75 documents *or* about 5 MB of raw text. Our 73 documents meet the document-count condition. The text size is below 5 MB, but each document is long (on average 6,730 words), so the corpus is large enough for meaningful TF-IDF statistics.

The text is realistic and noisy. Because it was exported from formatted web pages, it contains infobox fragments mixed into sentences, broken lines, URLs, citation markers such as [12], long reference lists with author names and journal titles, mixed casing and many domain-specific terms. Document

length varies strongly, from 39 words (*Tokenization*, a short disambiguation page) to 17,957 words (*Deep learning*).

Document structure and metadata

Text and metadata are kept separate. The raw text stays unchanged in `Dataset/`. The metadata is generated by `src/ingestion.py` and saved in `data/processed/corpus_metadata.csv`:

Field	Description
doc_id	Unique, stable ID (DOC001-DOC073, in file-name order; uploads get UPL001, ...)
title	Article title (first line of the file)
category	Practice-area label from data/metadata/category_mapping.csv
file_name	Original file in Dataset/ (traceability to the raw document)
source / source_url / revision_id	Wikipedia, and the exact revision URL taken from the 'Retrieved from' footer (the footer is removed from the text)
char_count / word_count	Length statistics
text	Main text content (in memory; read from the original file)

Table 1: Document fields

Category labels

The files did not come with labels, so we labelled them manually. We used the topic of each article's lead section, not only its file name. The mapping and a one-line reason for every document are in `data/metadata/category_mapping.csv`. Two articles (*Carlos Lehder*, a biography, and *Hippopotamuses in Colombia*) do not belong to any practice area. They are labelled *General (Off-domain)*. They stay in the corpus for browsing and recommendation, but are left out of classification and clustering evaluation, because two documents are too few to train and test a class. This leaves 71 labelled documents in five classes.

Category	Documents	Examples
Machine Learning	22	Backpropagation, Random forest, LSTM, GAN
Language & Search Technology	18	Stemming, Search engine, Machine translation
AI Systems & Cognition	11	ChatGPT, Turing test, Intelligent agent
Finance & Economics	11	Stock market, Financial crisis, Layoff
Computing & Infrastructure	9	Cloud computing, Data center, Cyberattack
General (Off-domain)	2	Carlos Lehder, Hippopotamuses in Colombia

Table 2: Category distribution

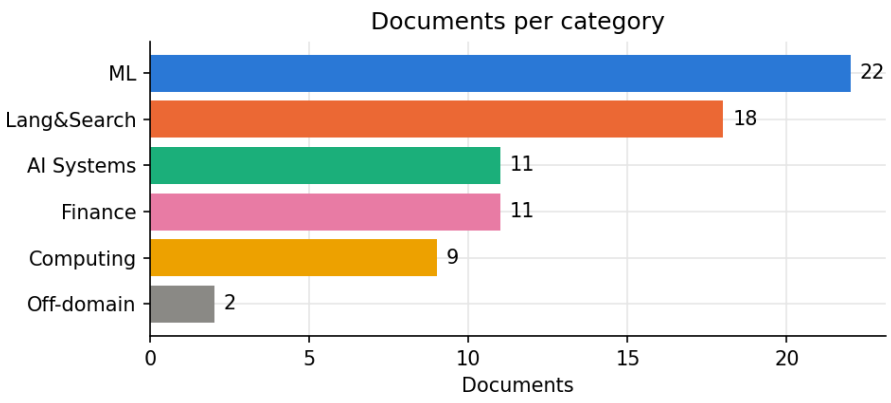


Figure 1: Documents per category

Some labels are debatable, and that is realistic. *Goldmines Telefilms* is a company profile of a film distributor that we put under Finance & Economics (business). *Conversation* and *Human intelligence* were put under AI Systems & Cognition. These borderline cases are visible later in the classification and clustering errors.

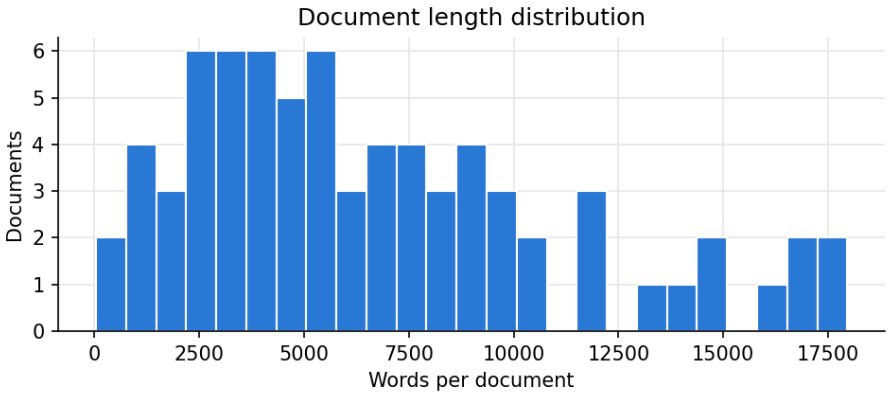


Figure 2: Distribution of document lengths (words)

5. Dataset Preparation and Duplicate Analysis

Ingestion reads every file as UTF-8 (removing the byte-order mark), normalises line endings, takes the first line as the title, and moves the *Retrieved from* footer into the metadata. Empty uploaded documents are skipped with a warning. Uploaded documents (.txt, .csv or .json) get unique IDs and are merged with the base corpus for the current session.

Duplicates are detected in two ways (`src/duplicates.py`):

- **Exact duplicates**: equal MD5 hash of the text after lowercasing and removing punctuation and extra whitespace.
- **Near duplicates**: Jaccard similarity of the sets of word 5-gram shingles, $J(A,B) = |A \cap B| / |A \cup B|$, and the cosine similarity of the TF-IDF vectors. A pair is flagged if $J \geq 0.5$ or cosine ≥ 0.9.

All 2,628 document pairs of the real corpus were compared. **No exact or near duplicates were found.** The most similar pair is *Deep learning* / *Neural network (machine learning)* with Jaccard 0.058 and cosine 0.503. These are related articles, not copies. Table 3 lists the closest pairs.

Document A	Document B	Jaccard (5-shingles)	TF-IDF cosine
Deep learning	Neural network (machine learning)	0.058	0.503
Long short-term memory	Recurrent neural network	0.052	0.291
Hallucination (artificial intelligence)	Machine learning	0.029	0.254
Convolutional neural network	Deep learning	0.024	0.366
Deep learning	Recurrent neural network	0.023	0.309
Convolutional neural network	Neural network (machine learning)	0.023	0.327

Table 3: Most similar document pairs in the corpus

Because the corpus has no duplicates, we did **not** add copies to the dataset files. Instead, (a) the application lets an analyst upload a file and see it flagged (Figure 3: re-uploading *Stemming.txt* is detected as an exact duplicate of DOC061), and (b) a separate, clearly labelled in-memory experiment measures the effect of duplicates (Section 16, Question 2).

Handling. Flagged documents form a duplicate group (union-find). The train/test split and cross-validation keep every group on one side, so a copy of a test document can never be in the training data. The recommender hides documents above the duplicate threshold, and the original files are never modified.

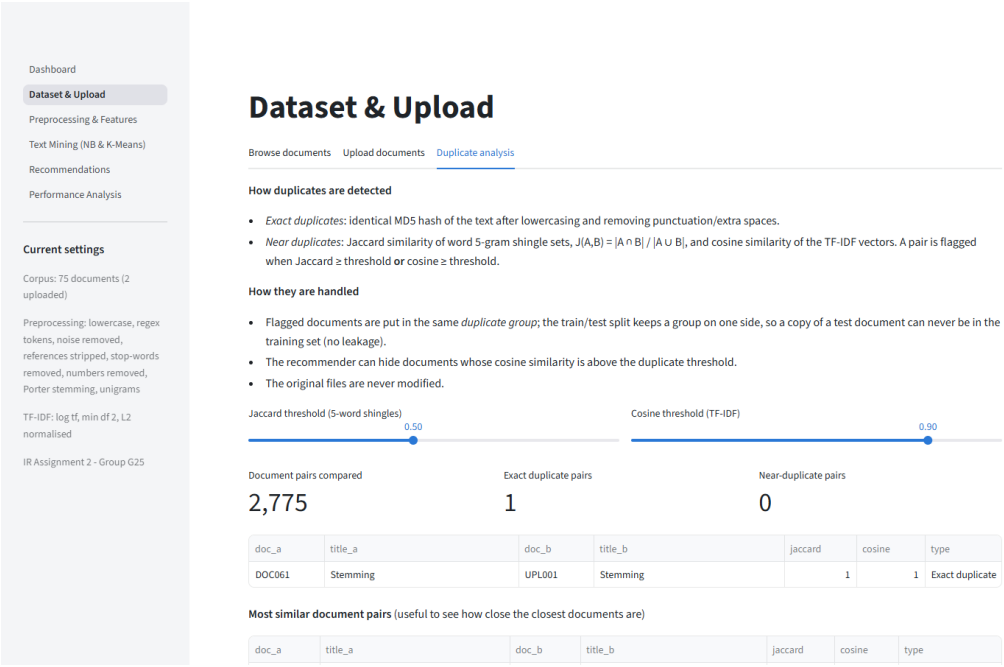


Figure 3: Duplicate analysis after uploading a copy of Stemming.txt (local run of the application)

6. System Architecture

The project is split into small modules. The Streamlit pages only handle the user interface and call the functions in `src/`. The experiment script calls the same functions, so the report and the application always show the same numbers.

```
Dataset/*.txt + uploads --> ingestion --> preprocessing --> TF-IDF + inverted
index
--> Naive Bayes classification --> metrics (accuracy, P, R, F1, confusion
matrix)
--> K-Means clustering --> Purity, Rand Index, 2D PCA plot
--> cosine similarity --> Top-K recommendations / text search
--> Streamlit pages (views/) and scripts/run_experiments.py -> report
```

Module	Responsibility
src/ingestion.py	Load files, IDs, metadata, uploads (.txt/.csv/.json), corpus statistics
src/preprocessing.py	Configurable cleaning, tokenization, stop-words, stemming, lemmatization, n-grams
src/features.py	Manual vocabulary, tf, df, idf, TF-IDF, L2 normalisation, cosine similarity
src/inverted_index.py	Postings lists and term-at-a-time cosine search
src/classification.py	Manual multinomial Naive Bayes, stratified split, cross-validation
src/clustering.py	Manual K-Means (k-means++/random init), PCA projection for plotting only
src/evaluation.py	Manual confusion matrix, accuracy/P/R/F1, Purity, Rand Index
src/duplicates.py	Exact and near-duplicate detection, duplicate groups
src/recommendation.py	Top-K content-based recommendation, explanation terms, precision@K
src/experiments.py	Preprocessing comparison, duplicate experiment, K sweep
src/app_state.py, views/, app.py	Streamlit session state, caching and the six pages
tests/	40 unit tests (pytest) including textbook examples

Table 4: Project modules

The application has six pages: **Dashboard** (business context and corpus overview), **Dataset & Upload** (browse, search, filter, upload, duplicate analysis), **Preprocessing & Features** (settings, before/after statistics, TF-IDF, inverted index, cosine similarity), **Text Mining** (Naive Bayes and K-Means with all algorithm steps), **Recommendations** and **Performance Analysis** (charts, comparison and experiments). Results are cached with a key built from a fingerprint of the corpus and all settings. Changing a setting or uploading a document recomputes the results, and unchanged settings reuse the cache.

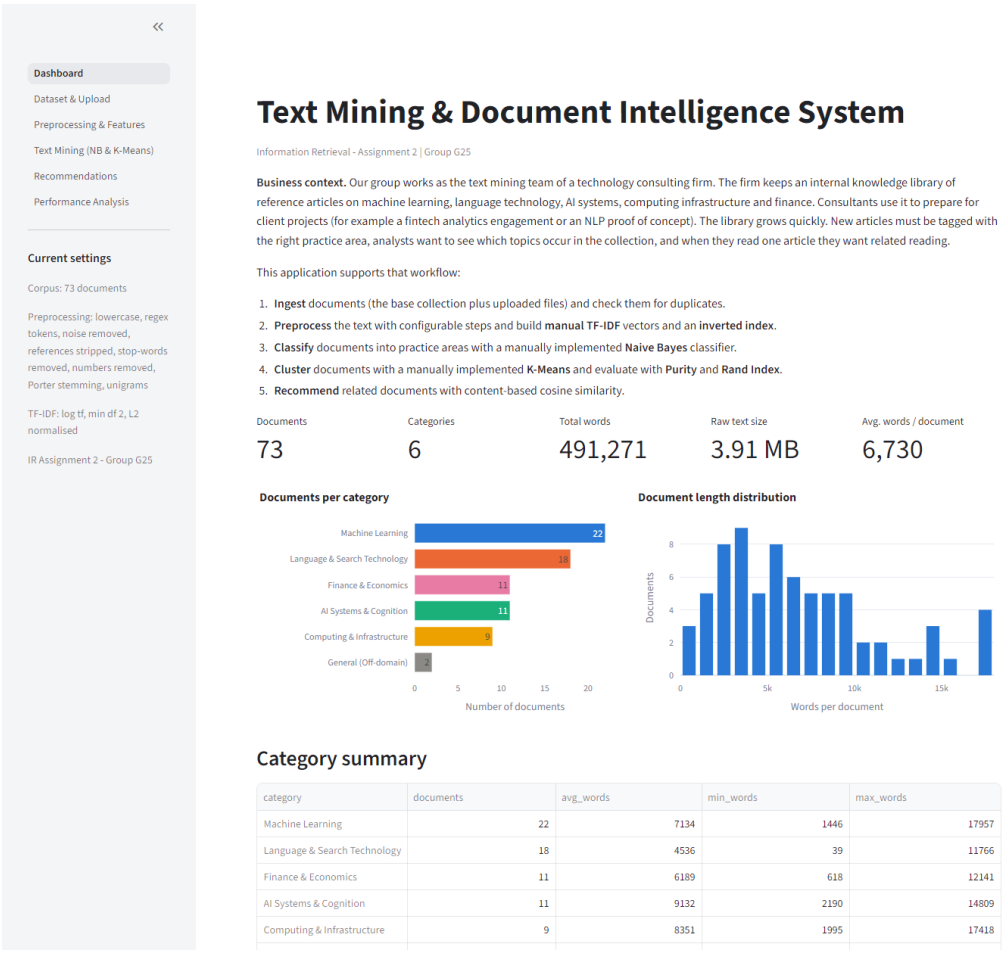


Figure 4: Dashboard page (local run)

7. Text Preprocessing

The preprocessing pipeline (`src/preprocessing.py`) is fully configurable from the Preprocessing page and applies its steps in this order:

- **Noise cleaning** (optional): remove the References / External links section, URLs and citation markers such as [12].
- **Lowercasing** (optional).
- **Tokenization**: a regular-expression tokenizer that keeps runs of letters/digits (default), or a plain whitespace split for comparison.
- **Filtering**: stop-word removal (NLTK English list plus a few wiki words such as 'retrieved', 'archived'), removal of tokens containing digits, minimum token length 2.
- **Word normalisation**: none, Porter stemming (NLTK), or WordNet lemmatization (NLTK; a small rule-based fallback is used if WordNet data is not installed).
- **N-grams**: unigrams, unigrams + bigrams, or up to trigrams (joined with '_', built from the filtered tokens).

The default setting is: lowercase, regex tokens, noise and references removed, stop-words and numbers removed, Porter stemming, unigrams. Table 5 and Figures 5-6 compare the corpus before preprocessing (plain whitespace tokens of the raw text) and after the default pipeline.

Statistic	Before	After (default)	Change
Vocabulary size	92,721	13,081	-85.9%
Total tokens	491,271	191,513	-61.0%
Average tokens per document	6,730	2,623	-61.0%
TF-IDF features (df >= 2)	-	5,446	

Table 5: Vocabulary before and after preprocessing

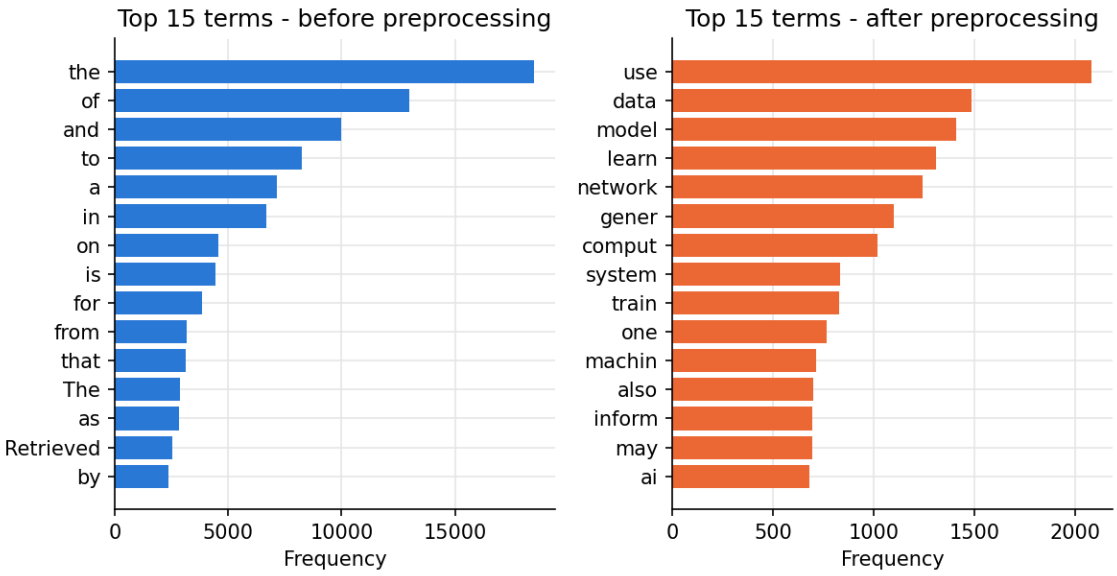


Figure 5: Most frequent terms before and after preprocessing. Before, the list is dominated by stop-words and wiki noise ('Retrieved', 'Archived', 'original'). After, it shows domain terms.

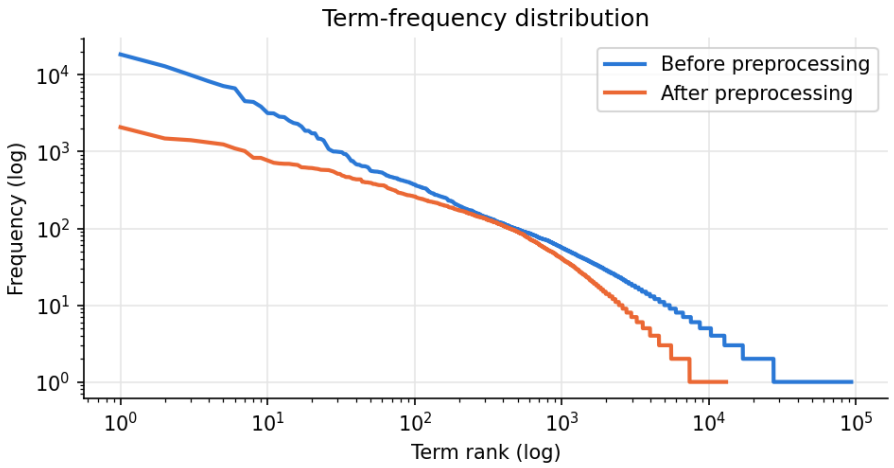


Figure 6: Rank-frequency (Zipf) curves before and after preprocessing

Before preprocessing the most frequent terms are *the, of, and, to, a, in*. After preprocessing they are *use, data, model, learn, network, gener, comput, system*. Very general words such as 'use', 'one' and 'also' remain, because they are not in the NLTK stop-word list. However, they occur in almost every document, so their IDF, and therefore their TF-IDF weight, is close to zero.

8. Manual TF-IDF and Inverted Index

TF-IDF is implemented from scratch in `src/features.py`. For N documents:

```
tf(t,d) = number of times term t occurs in document d
df(t) = number of documents that contain t
w_tf(t,d) = 1 + log10(tf(t,d)) if tf > 0 (option: raw tf)
idf(t) = log10(N / df(t))
w(t,d) = w_tf(t,d) * idf(t)
w'(t,d) = w(t,d) / sqrt( sum_t' w(t',d)^2 ) (L2 length normalisation)
cos(d1,d2) = sum_t w'(t,d1) * w'(t,d2)
```

We use the logarithmic tf so that a word repeated 100 times does not count 100 times more than a word that appears once. A term that appears in every document gets idf = 0 and no weight. The vocabulary contains only terms with df ≥ 2 (configurable), which removes most typos and one-off tokens from the reference lists. Documents are stored as sparse dictionaries {term: weight} and also as a dense NumPy matrix for K-Means. L2 normalisation makes every vector unit length, so long and short documents can be compared fairly, and cosine similarity becomes a dot product.

Inverted index. For every term the index stores a postings list of (document, raw tf, TF-IDF weight), sorted by document (`src/inverted_index.py`). With the default settings it has 5,446 terms and 54,210 postings (average postings length 10.0, longest 71). Free-text queries are answered term-at-a-time. For each query term we walk its postings list and add query weight x document weight to the document's score. Because both vectors are normalised, the score is exactly the cosine similarity, and only documents that share at least one term with the query are visited. A unit test checks that index scores equal the direct cosine computation.

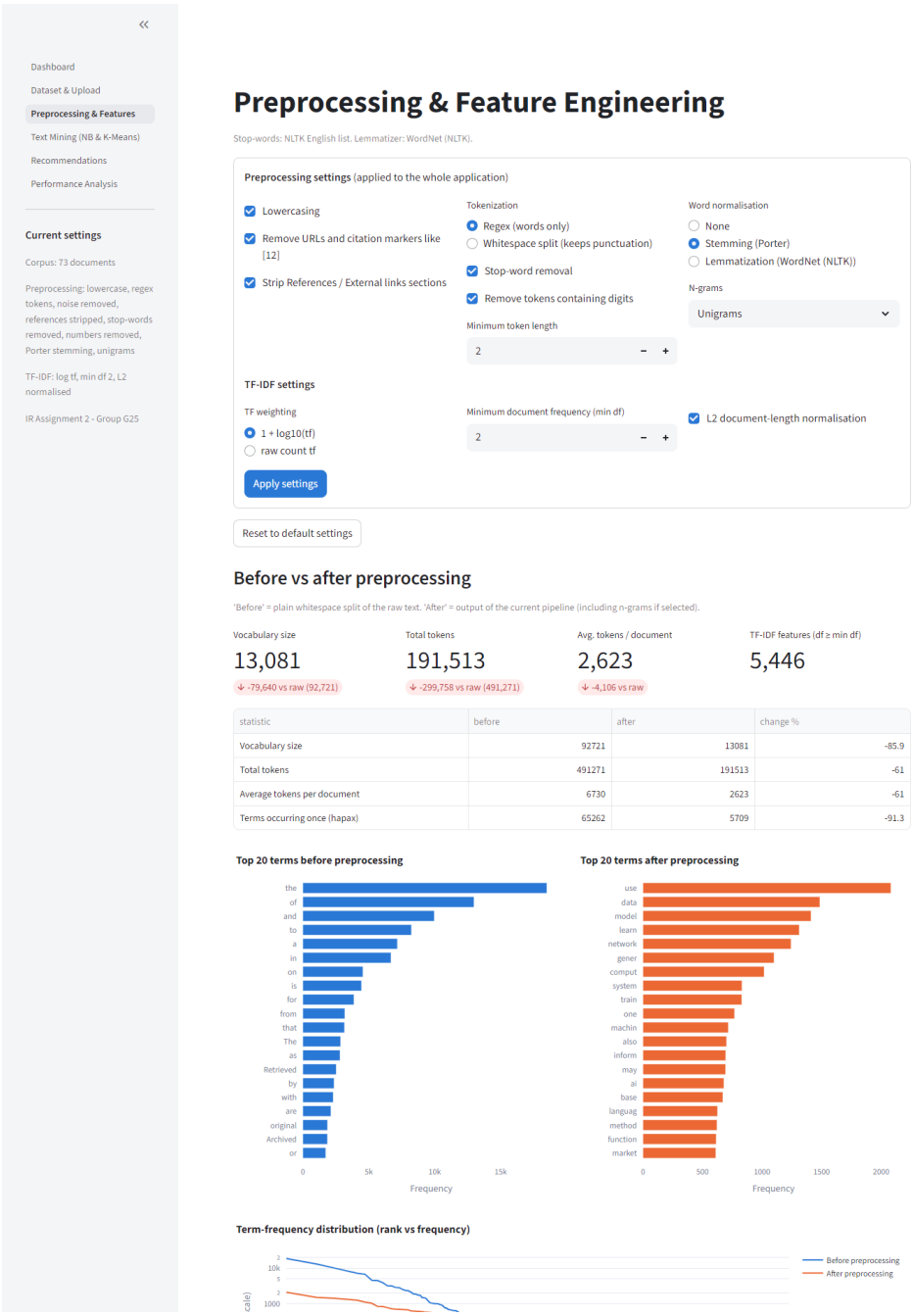


Figure 7: Preprocessing page with settings and before/after statistics (local run)

9. Naive Bayes Classification

We implemented a multinomial Naive Bayes classifier with Laplace (add-alpha) smoothing (`src/classification.py`). It is trained on the term counts of the preprocessed documents:

```
P(c) = N_c / N
P(t | c) = (count(t,c) + alpha) / (sum_t' count(t',c) + alpha * |V|)
score(c|d) = log P(c) + sum_{t in d} tf(t,d) * log P(t|c)
prediction = argmax_c score(c|d)
```

Logarithms prevent numerical underflow when thousands of small probabilities are multiplied. The vocabulary V, the priors and the likelihoods are computed from the **training documents only**. Words of a test document that never appeared in training are ignored, so the test set cannot influence the model. The implementation reproduces the worked example 13.1 of Manning et al. (2008) exactly

(unit test).

Data split. The 71 labelled documents are split 70/30 with a stratified split (seed 42), giving 50 training and 21 test documents with every class present in both sets. Duplicate groups are kept on one side of the split. Because the test set is small, we also report stratified 5-fold cross-validation, in which every document is tested exactly once.

Metrics (implemented in `src/evaluation.py`): accuracy, and per class precision = TP/(TP+FP), recall = TP/(TP+FN), F1 = 2PR/(P+R). We report the macro average (every class counts equally, which matters because our classes have different sizes) and the weighted average (weighted by class support).

10. K-Means Clustering

K-Means is implemented from scratch in `src/clustering.py` and runs on the L2-normalised TF-IDF vectors of the 71 labelled documents. The steps shown in the application are:

- **Initialisation**: k-means++ (default). The first centroid is a random document, and each next centroid is a document chosen with probability proportional to its squared distance from the nearest existing centroid. Plain random initialisation is also available.
- **Assignment**: each document goes to the centroid with the smallest squared Euclidean distance. For unit-length vectors $||x - c||^2 = 1 + ||c||^2 - 2\ x.c$, so this ranks centroids by cosine-like similarity.
- **Update**: each centroid becomes the mean vector of its documents. An empty cluster is re-seeded with the document that is farthest from its current centroid.
- **Stopping criterion**: stop when no document changes cluster or the largest centroid movement is below 10^{-6}, or after the maximum number of iterations (100).
- The algorithm is run 10 times with different seeds, and the run with the lowest SSE (within-cluster sum of squared distances) is kept.

With K = 5 (the number of true categories), the best run started from the seed documents *Amazon Web Services, Long short-term memory, Financial crisis, Philosophy of artificial intelligence, Autoencoder*. It converged after 2 iterations (SSE 56.309). All 10 runs converged within 2-3 iterations. TF-IDF vectors are very high-dimensional and sparse, and documents are almost orthogonal to documents of other topics, so k-means++ seeds usually lead to a stable assignment after the first update.

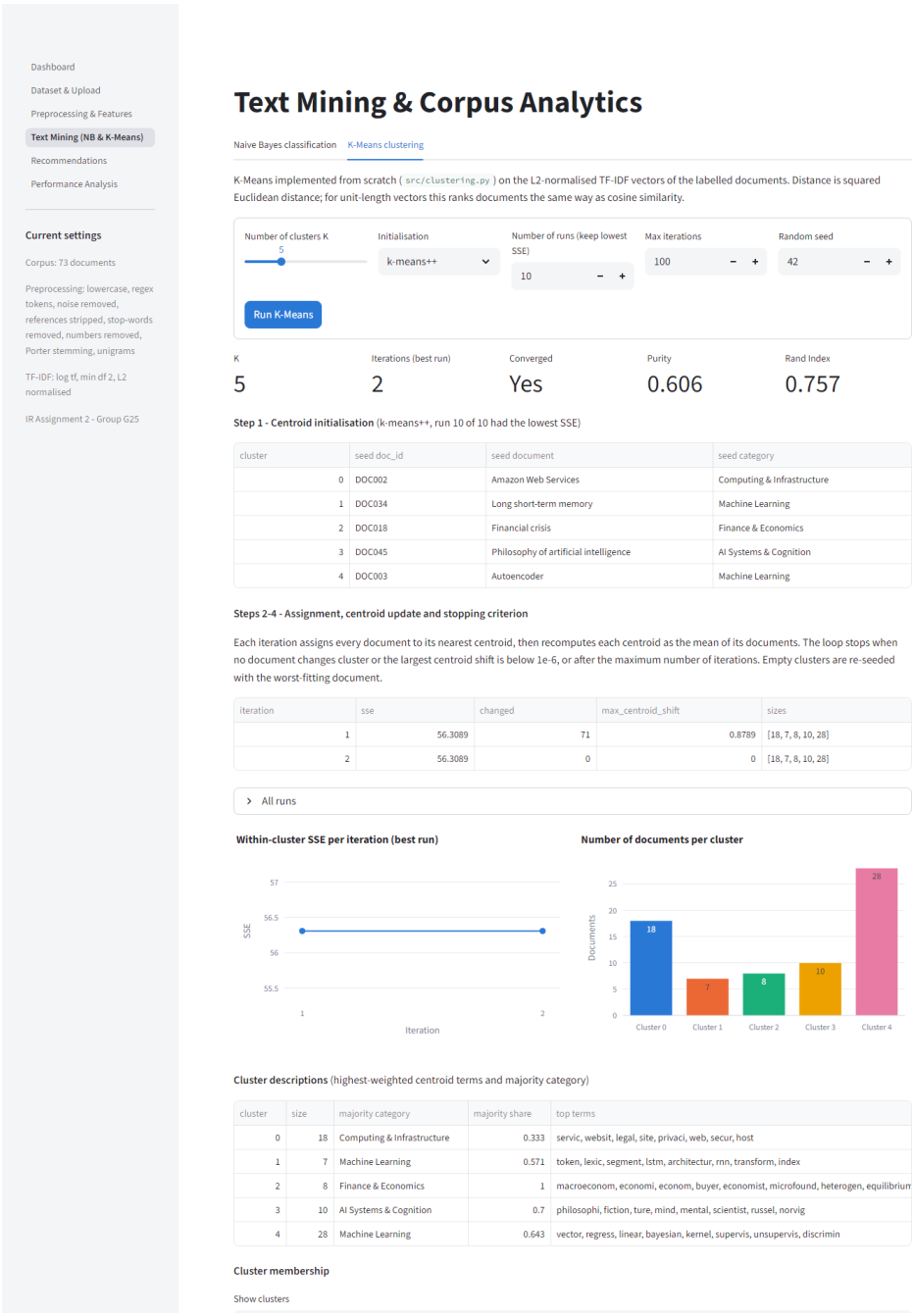


Figure 8: K-Means page: initial seeds, iterations, cluster sizes and descriptions (local run)

11. Clustering Evaluation

Purity and Rand Index are implemented without any library (`src/evaluation.py`):

```
Purity = (1/N) * sum_k max_j |cluster_k intersect class_j|
RI = (TP + TN) / (TP + FP + FN + TN)
TP: pair in same cluster and same class FP: same cluster, different class
FN: different cluster, same class TN: different cluster and class
```

The pair counts are computed from the cluster-class contingency table: TP + FP = Σ_k C(n_k, 2) and TP = $\Sigma_{k,j}$ C(n_{kj}, 2). A second brute-force version that loops over all pairs is used in the unit tests to check the fast version. Both metrics reproduce the textbook example of Manning et al. (Figure 16.4: purity 0.71, RI 0.68). Purity measures how 'clean' each cluster is. Rand Index measures agreement on all document pairs. Purity always increases as K grows (with K = N it is 1), so it must be compared at a

fixed K.

12. Document Recommendation

We implemented **content-based** recommendation (`src/recommendation.py`). For a selected document d, every other document d' is scored by the cosine similarity of their TF-IDF vectors, and the K highest are returned. The selected document is always excluded. Documents with cosine ≥ 0.9 can be hidden as near-duplicates. For each recommendation the application shows the document ID, title, category, similarity score, whether the category matches, and the *shared terms*: the terms with the largest product w(t,d)·w(t,d'), which explain the score. Users can also type free text, which is scored through the inverted index.

As a simple offline quality measure we use **category precision@5**, the share of the top-5 recommendations that have the same category as the selected document, averaged over all labelled documents. With the default settings it is **0.721**.

13. Experimental Results

13.1 Naive Bayes

Metric	Test split (21 docs)	5-fold CV (71 docs)
Accuracy	0.857	0.831
Macro precision	0.906	0.834
Macro recall	0.827	0.773
Macro F1	0.844	0.777
Weighted F1	0.854	0.817

Table 6: Naive Bayes evaluation (default settings, alpha = 1)

Class	Precision	Recall	F1	Support
AI Systems & Cognition	0.750	1.000	0.857	3
Computing & Infrastructure	1.000	0.667	0.800	3
Finance & Economics	1.000	0.667	0.800	3
Language & Search Technology	1.000	0.800	0.889	5
Machine Learning	0.778	1.000	0.875	7

Table 7: Per-class metrics on the test split

Doc ID	Title	Actual	Predicted	
DOC002	Amazon Web Services	Computing & Infrastructure	Computing & Infrastructure	
DOC006	ChatGPT	AI Systems & Cognition	AI Systems & Cognition	
DOC014	Data center	Computing & Infrastructure	Computing & Infrastructure	
DOC015	Data structure	Computing & Infrastructure	Machine Learning	wrong
DOC017	Deep learning	Machine Learning	Machine Learning	
DOC018	Financial crisis	Finance & Economics	Finance & Economics	
DOC020	Generative adversarial network	Machine Learning	Machine Learning	
DOC027	Information retrieval	Language & Search Technology	Language & Search Technology	
DOC030	Intelligent agent	AI Systems & Cognition	AI Systems & Cognition	
DOC033	Lemmatization	Language & Search Technology	Language & Search Technology	
DOC037	Markov decision process	Machine Learning	Machine Learning	
DOC038	Money market fund	Finance & Economics	Finance & Economics	
DOC041	Neural machine translation	Language & Search Technology	Machine Learning	wrong
DOC042	Neural network (machine learning)	Machine Learning	Machine Learning	
DOC051	Representative agent	Finance & Economics	AI Systems & Cognition	wrong
DOC056	Speaker recognition	Language & Search Technology	Language & Search Technology	
DOC059	Statistical classification	Machine Learning	Machine Learning	
DOC064	Support vector machine	Machine Learning	Machine Learning	
DOC069	Turing test	AI Systems & Cognition	AI Systems & Cognition	
DOC070	Unsupervised learning	Machine Learning	Machine Learning	
DOC073	Word-sense disambiguation	Language & Search Technology	Language & Search Technology	

Table 8: Actual and predicted labels of all test documents

3 of 21 test documents are misclassified: *Data structure* (Computing & Infrastructure predicted as Machine Learning); *Neural machine translation* (Language & Search Technology predicted as Machine Learning); *Representative agent* (Finance & Economics predicted as AI Systems & Cognition). Each error makes sense. *Neural machine translation* is mostly about neural network architectures, *Data structure* uses algorithmic vocabulary that is common in the ML articles, and *Representative agent* shares the word 'agent' with the AI agent articles. Machine Learning, the largest class, has perfect recall but lower precision, because borderline documents are pulled into it.

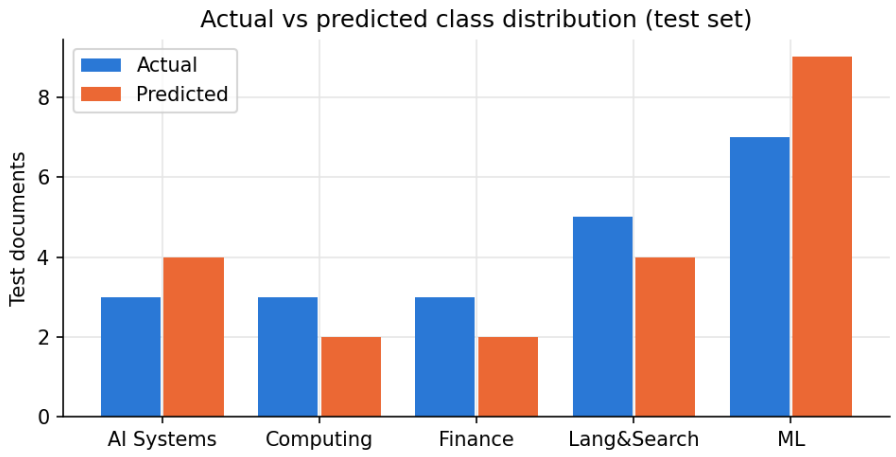


Figure 9: Actual vs predicted class distribution on the test set

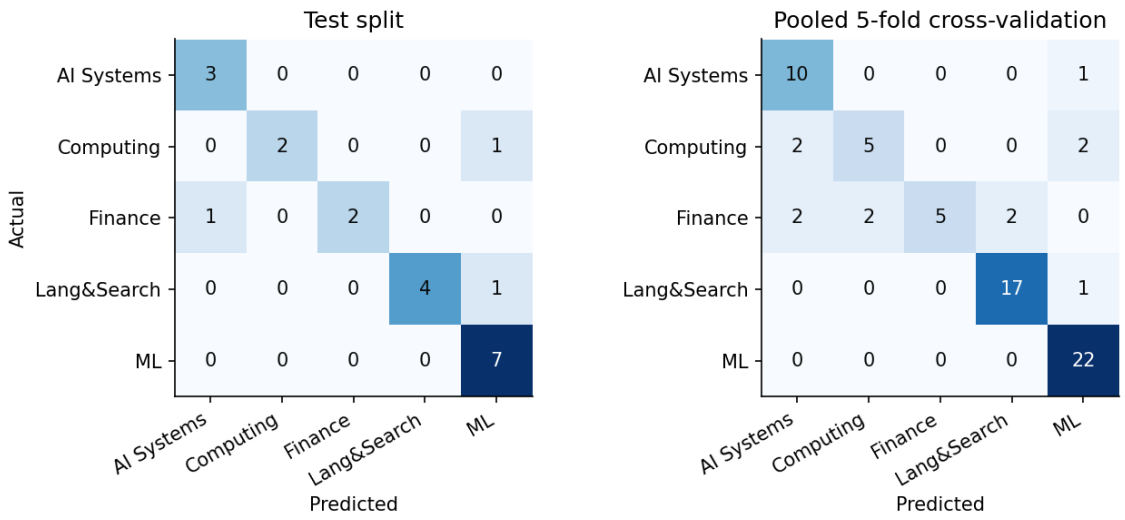


Figure 10: Confusion matrices: test split (left) and pooled 5-fold cross-validation (right)

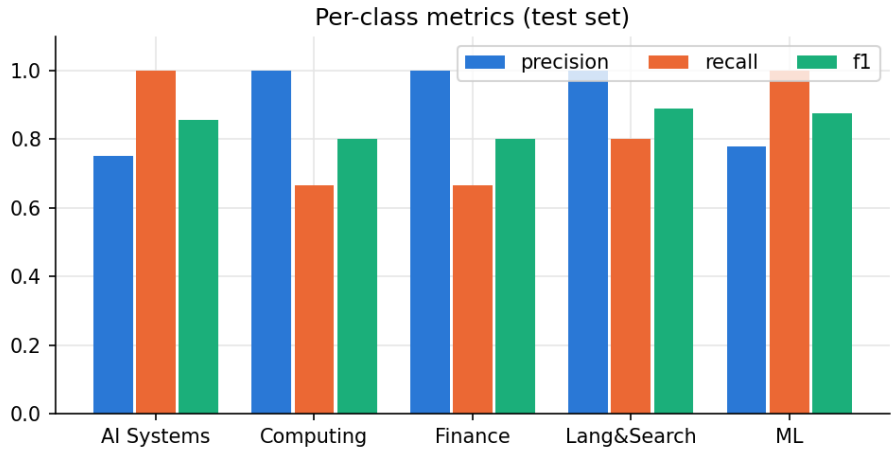


Figure 11: Precision, recall and F1 per class (test split)

In cross-validation the weakest classes are **Finance & Economics** (recall 0.45) and **Computing & Infrastructure** (recall 0.56). These are the smallest and most heterogeneous classes. For example, Finance contains a film company profile and an article on cryptocurrency crime. The most indicative Naive Bayes terms show this too. For Finance they include *telugu, tamil, currenc, cryptocurr, trade, stock, exchang, bhojpuri*, where 'telugu', 'tamil' and 'bhojpuri' come only from the Goldmines Telefilms article.

13.2 K-Means

Cluster	Size	Composition (true categories)	Top centroid terms
0	18	Computing 6, Language 5, AI 4, Finance 3	servic, websit, legal, site, privaci, web
1	7	Machine 4, Language 3	token, lexic, segment, lstm, architectur, rnn
2	8	Finance 8	macroeconom, economi, econom, buyer, economist, microfound
3	10	AI 7, Language 2, Computing 1	philosophi, fiction, ture, mind, mental, scientist
4	28	Machine 18, Language 8, Computing 2	vector, regress, linear, bayesian, kernel, supervis

Table 9: K-Means clusters (K = 5)

Purity = 0.606 and **Rand Index = 0.757** (TP 275, FP 350, FN 255, TN 1605 over 2,485 pairs). Some clusters are clean. The economics-theory cluster is 100% Finance, and the cluster about philosophy

of mind and the Turing test is mostly AI Systems. Others mix categories. The largest cluster contains the statistical / ML articles together with 8 language-technology articles that use the same vocabulary (vectors, models, training). Another cluster collects web-, service- and security-oriented articles from four categories. K-Means groups documents by shared vocabulary, which does not always match our human-defined practice areas.

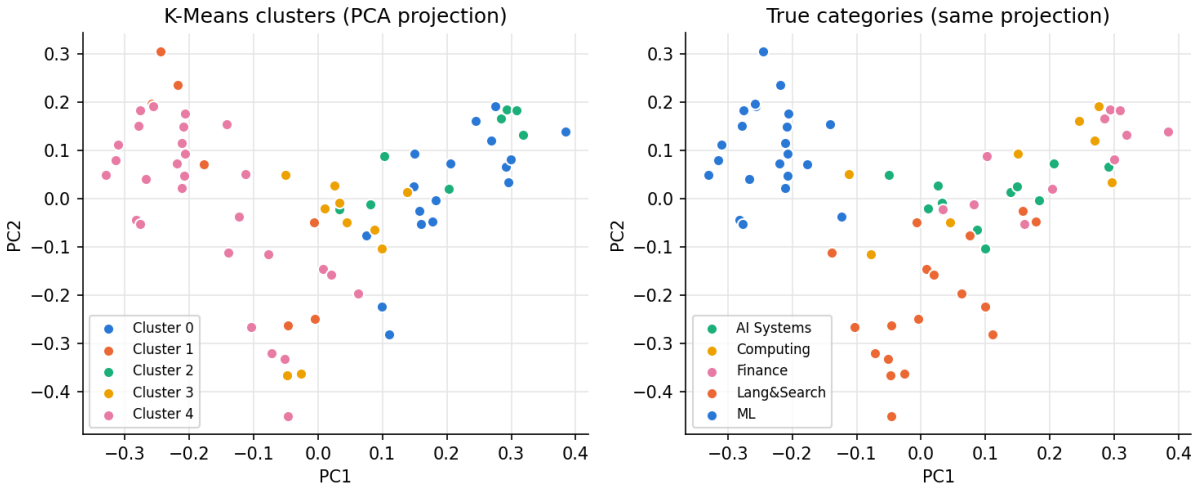


Figure 12: 2D PCA projection of the TF-IDF vectors, coloured by K-Means cluster (left) and true category (right). PCA is used only for plotting (2 components explain 7.4% of the variance).

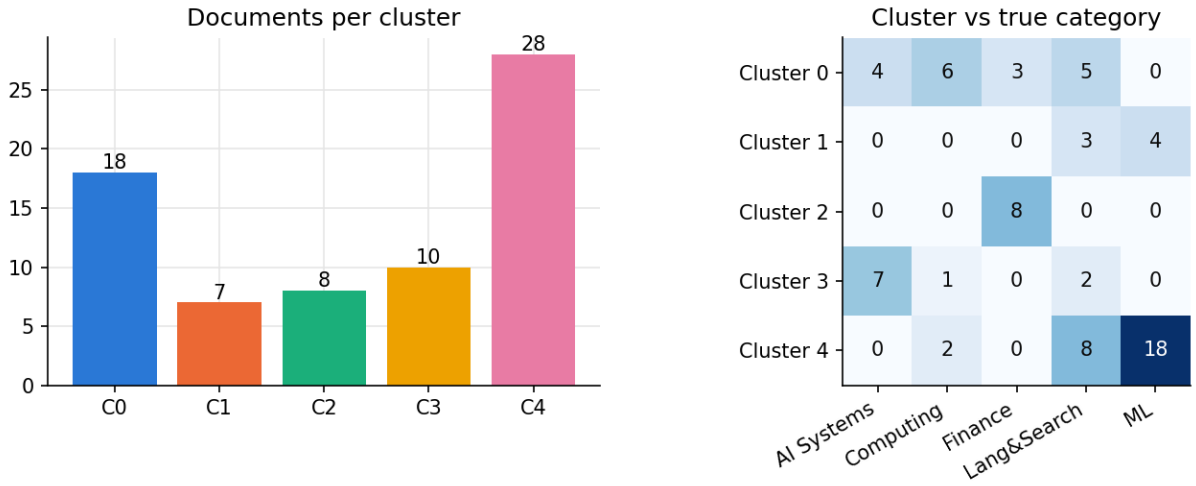


Figure 13: Cluster sizes and cluster vs category contingency table

Effect of K and initialisation. Table 10 shows Purity, Rand Index and SSE for K = 2-10. SSE decreases steadily and has no clear elbow. Purity generally rises with K because smaller clusters are purer, but it is not monotonic because each K is a separate local optimum. Running the same configuration with five different seeds gave a mean purity of 0.648 and mean RI of 0.774 at K = 5. Random initialisation reached purity 0.690 / RI 0.820, even though its best SSE was higher. A lower SSE does not guarantee better agreement with the human labels.

K	Purity	Rand Index	SSE
2	0.465	0.625	60.22
3	0.549	0.699	58.99
4	0.634	0.743	57.59
5	0.606	0.757	56.31
6	0.761	0.826	55.16
7	0.648	0.775	53.77
8	0.718	0.825	52.55
9	0.803	0.849	51.50
10	0.704	0.787	50.52

Table 10: K-Means results for different K (seed 42)

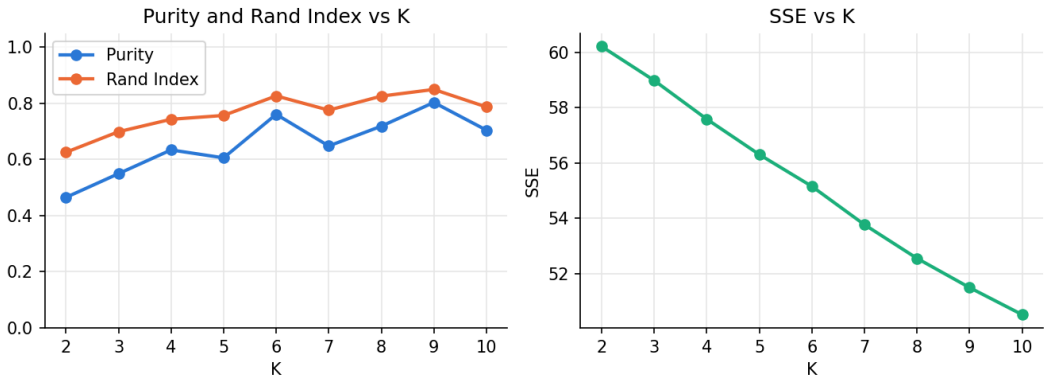


Figure 14: Purity, Rand Index and SSE for different K

13.3 Recommendations

Rank	Doc ID	Title	Category	Cosine	Shared terms
1	DOC047	Recurrent neural network	Machine Learning	0.291	rnn, lstm, gru, gate, ctc, forget
2	DOC017	Deep learning	Machine Learning	0.253	lstm, timit, rnn, forget, grave, highway
3	DOC042	Neural network (machine learning)	Machine Learning	0.235	neuroevolut, rnn, highway, schmidhub, lstm, forget
4	DOC057	Speech recognition	Language & Search Technology	0.189	ctc, grave, lstm, rnn, switchboard, fred
5	DOC011	Convolutional neural network	Machine Learning	0.180	hyperbol, explod, rnn, vanish, convolut, tangent

Table 11: Top-5 recommendations for 'Long short-term memory' (DOC034, Machine Learning)

Rank	Doc ID	Title	Category	Cosine	Shared terms
1	DOC019	Foreign exchange market	Finance & Economics	0.388	forex, trader, fx, currenc, otc, bid
2	DOC018	Financial crisis	Finance & Economics	0.370	crash, recess, debt, investor, herd, panic
3	DOC038	Money market fund	Finance & Economics	0.333	sec, debt, banker, investor, dividend, bond
4	DOC065	Technical analysis	Finance & Economics	0.316	emh, dow, trader, investor, stock, irrat
5	DOC014	Data center	Computing & Infrastructure	0.229	floor, indirect, household, tax, ownership, retir

Table 12: Top-5 recommendations for 'Stock market' (DOC062, Finance & Economics)

Rank	Doc ID	Title	Category	Cosine	Shared terms
1	DOC051	Representative agent	Finance & Economics	0.287	macroeconom, microfound, heterogen, equilibrium, economist, economi
2	DOC062	Stock market	Finance & Economics	0.162	taxat, household, pension, seller, macroeconom, redistribut
3	DOC055	Software agent	AI Systems & Cognition	0.115	behalf, deleg, heterogen, agent, buyer, persist
4	DOC019	Foreign exchange market	Finance & Economics	0.113	macroeconom, pension, seller, economi, behalf, maker
5	DOC018	Financial crisis	Finance & Economics	0.099	economi, incent, economist, equilibrium, econom, oblig

Table 13: Top-5 recommendations for 'Agent (economics)' (DOC001, Finance & Economics)

The recommendations are mostly sensible. *Long short-term memory* leads to recurrent networks, deep learning and speech recognition, which uses LSTMs. *Stock market* leads to the other finance articles. Two weaknesses are also visible. For *Stock market*, the 5th result is *Data center*, matched on weak terms such as 'floor', 'tax' and 'ownership'. For *Agent (economics)*, *Software agent* appears because of the ambiguous word 'agent'. Free-text search for "How do recurrent neural networks handle long sequences in speech recognition?" returned *Recurrent neural network* (0.089), *Long short-term memory* (0.086), *Neural machine translation* (0.084), *Pattern recognition* (0.071), *Speech recognition* (0.063).

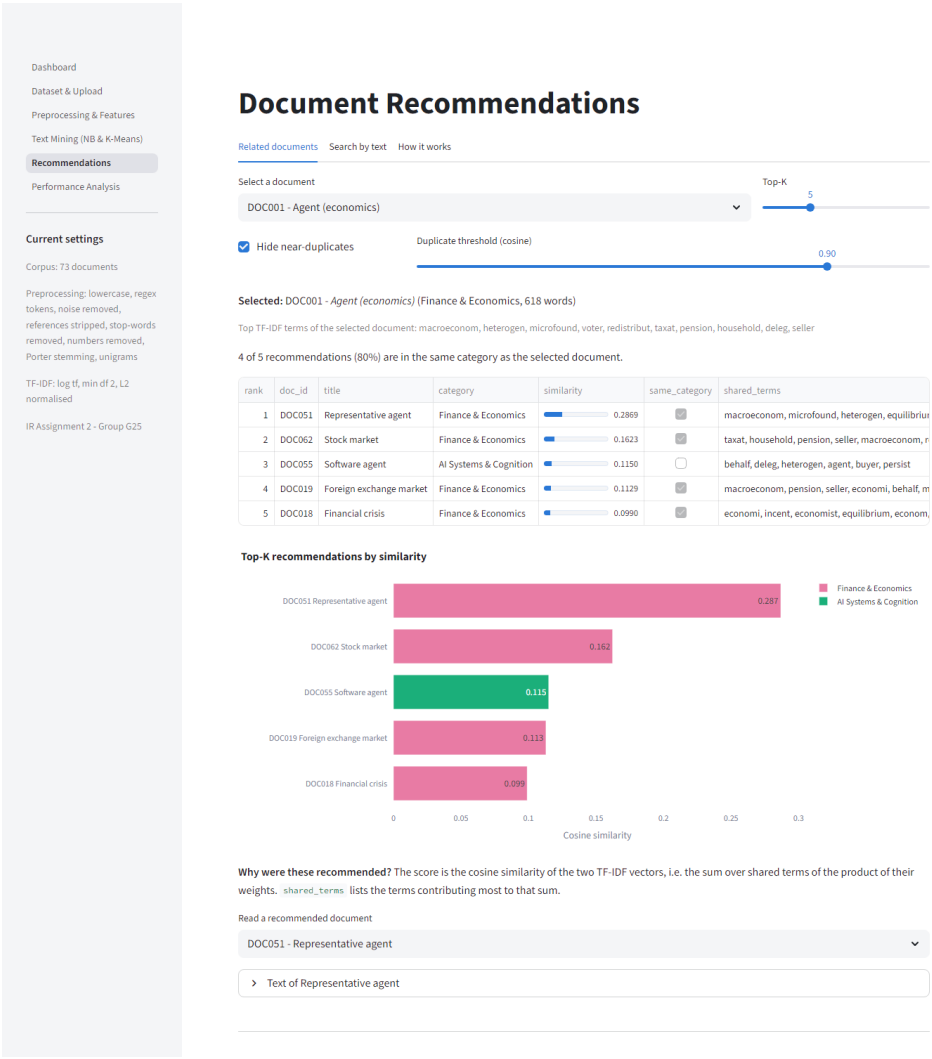


Figure 15: Recommendation interface (local run)

14. Visualizations

The application contains all required visualisations. The **Performance Analysis** page shows the actual vs predicted class distribution, the confusion matrices (test split and cross-validation), and the per-class metrics. Its Clustering tab shows the 2D scatter plot (switchable between cluster colours and true-category colours), the cluster-size bar chart, the cluster vs category heatmap and the cluster membership table (document ID, title, true label, assigned cluster). The Comparison tab shows Purity and Rand Index next to the Naive Bayes metrics with an interpretation, and the Experiments tab shows the experiments below. The Preprocessing page shows the term-frequency charts, the Zipf curves, the document-frequency histogram and a document-similarity heatmap. The Recommendations page shows the score chart.

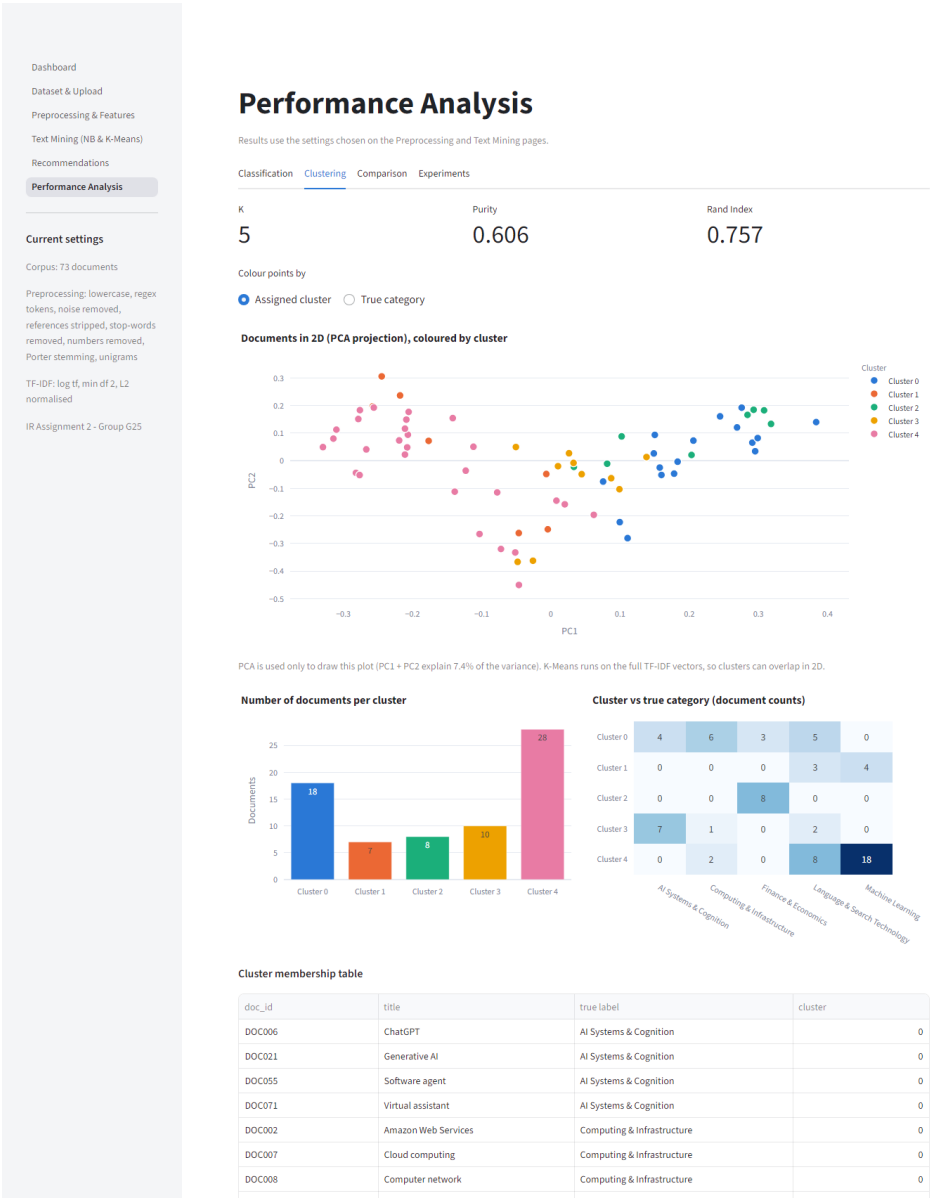


Figure 16: Clustering visualisations on the Performance Analysis page (local run)

15. Comparative Analysis

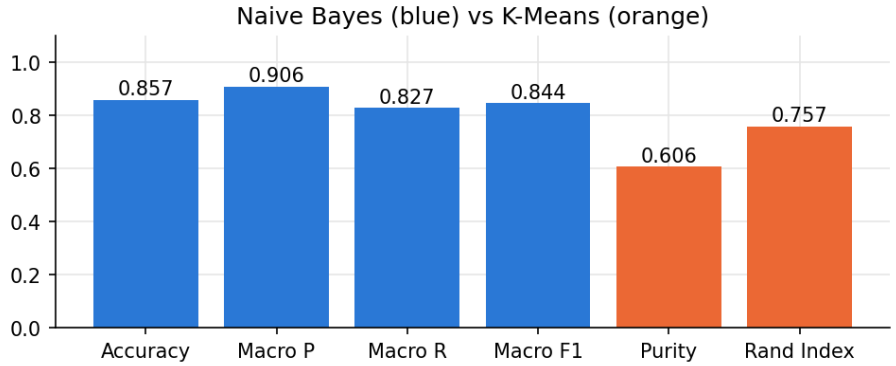


Figure 17: Naive Bayes metrics (test split) next to K-Means Purity and Rand Index

Aspect	Naive Bayes	K-Means
Learning type	Supervised: uses the category labels	Unsupervised: never sees the labels
Output	One of the 5 known categories per document	K groups without names
Evaluation	Accuracy, precision, recall, F1, confusion matrix	Purity, Rand Index (labels used only for evaluation)
Result (default)	Accuracy 0.857, macro F1 0.844 (CV: 0.831 / 0.777)	Purity 0.606, RI 0.757 (5-seed mean 0.648 / 0.774)
Main errors	Cross-domain documents pulled into the largest class	Clusters follow shared vocabulary (e.g. web/security, sequence models) rather than practice areas
Use in the library	Auto-tagging new articles	Discovering themes, checking the category scheme

Table 14: Naive Bayes vs K-Means

The two methods solve different problems, so their numbers cannot be compared directly. Accuracy checks whether each document received the correct label. Purity checks whether each cluster is dominated by one label, and Rand Index checks pairwise agreement. Naive Bayes clearly fits our labels better, because it learns them. K-Means reaches a purity of about 0.6-0.65 without any labels. That is useful, but not reliable enough to replace manual tagging.

16. Compulsory Inference and Discussion

Question 1: How did the choice of preprocessing steps affect the vocabulary, classification, clustering and recommendations?

We evaluated seven preprocessing settings. Each setting changes one step compared with an earlier setting, as its name shows. Every setting was run on the same corpus with the same split and seeds (Table 15, Figure 18).

Setting	Vocab	TF-IDF feat.	Terms/ doc	NB acc (test)	NB acc (CV)	NB F1 (CV)	Purity*	RI*	P@5
A. Minimal (lowercase + tokens only)	47,994	16,170	8,279	0.810	0.859	0.843	0.628	0.733	0.701
B. A + noise/reference/number cleaning	19,234	8,725	4,098	0.857	0.817	0.763	0.654	0.789	0.718
C. B + stop-word removal	19,095	8,594	2,623	0.857	0.831	0.783	0.654	0.787	0.718
D. C + Porter stemming (default)	13,081	5,446	2,623	0.857	0.831	0.777	0.648	0.774	0.721
E. C + lemmatization	14,978	6,323	2,623	0.857	0.831	0.777	0.642	0.767	0.724
F. D + bigrams	151,108	20,152	5,246	0.810	0.803	0.755	0.682	0.802	0.746
G. D without length normalization	13,081	5,446	2,623	0.857	0.831	0.777	0.392	0.380	0.721

Table 15: Effect of preprocessing (* K-Means K = 5, mean over 5 seeds; P@5 = category precision@5)

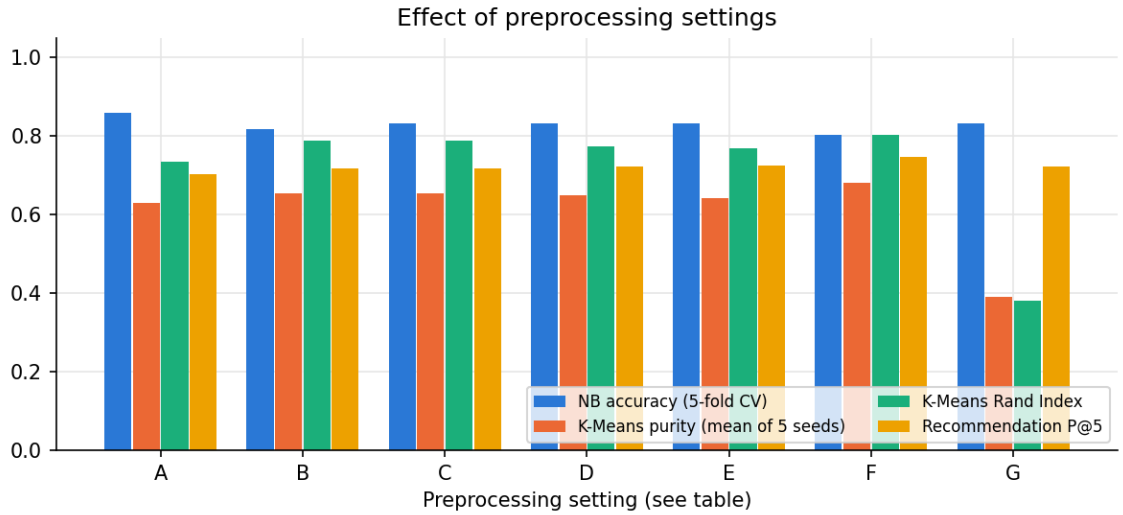


Figure 18: Effect of preprocessing settings on the three tasks

- **Vocabulary.** Cleaning (removing reference sections, URLs and numbers) has the largest effect. It cuts the vocabulary from 47,994 to 19,234 terms and halves the terms per document (8,279 to 4,098), so about half of the raw text is reference lists. Stop-word removal hardly changes the vocabulary (only ~150 words) but removes 36% of all tokens. Stemming reduces the vocabulary to 13,081 (-31%), and lemmatization is less aggressive (14,978). Adding bigrams multiplies the vocabulary to 151,108.
- **Classification.** Naive Bayes was robust. Cross-validated accuracy stayed between 0.803 and 0.859, and on the 21-document test split the settings differ by only one document. Surprisingly, the minimal setting had the highest CV accuracy (0.859). Naive Bayes can use the noisy tokens: reference lists contain names of authors, journals and conferences that are strongly topic-specific. Stemming and lemmatization did not change accuracy, and bigrams slightly reduced it (0.803). Bigrams add many rare features that make the probability estimates noisier for a training set of only ~57 documents per fold.
- **Clustering** was more sensitive. Cleaning improved purity from 0.628 to 0.654 and RI from 0.733 to 0.789. Without cleaning, the shared citation vocabulary makes unrelated documents look similar. Bigrams gave the best clusters (purity 0.682, RI 0.802) because phrases like 'neural_network' or 'stock_market' are more topic-specific than single words. The most important factor was **length normalisation**. Without it, purity fell to 0.392 and RI to 0.380. With the seed-42 run, 57 of 71 documents ended in one cluster while the three longest articles formed a cluster of their own: Euclidean distance then measures document length, not topic.

- **Recommendations** improved steadily with cleaning, stemming/lemmatization and bigrams (P@5 from 0.701 to 0.746). Normalisation does not change recommendations (setting G = D), because cosine similarity already divides by the vector lengths.
- **Conclusion:** noise cleaning and length normalisation matter most. Stemming mainly makes the model smaller (31% fewer features at the same quality). Bigrams help the similarity-based tasks but not Naive Bayes on this small training set. We kept stemmed unigrams as the default because they are fast and their results are easy to explain. Analysts can switch to bigrams in the application.

Question 2: How can duplicate or near-duplicate documents affect classification, clustering and recommendation? How did we detect or handle them?

Detection in our data. Section 5 shows that the real corpus has no exact or near duplicates (maximum Jaccard 0.058). To measure the effect of duplicates anyway, `src/experiments.py` adds, in memory only, an exact copy and an edited copy (10% of the words randomly deleted) of 10 labelled documents, labelled *[SIMULATED]*. This gives 93 documents. The dataset files are not changed. Results:

Aspect	Original corpus	With duplicates, no handling	With duplicates, our handling
Detection	-	-	30/30 copy pairs found, 0 false positives
NB accuracy (mean of 20 splits)	0.833	0.883 (on average 7.9 test documents have a copy in training)	0.828 (duplicate groups kept together)
Share of top-5 recommendation slots taken by copies	-	40.0%	0.0% (near-duplicates hidden)
Category precision@5	0.721	0.796 (inflated)	-
K-Means purity / RI	0.606 / 0.757	0.549 / 0.698	0.606 / 0.778 (copies removed)

Table 16: Duplicate-injection experiment

- **Classification:** with a normal random split, copies of test documents end up in the training set. The classifier then 'recognises' them, and accuracy rises from 0.833 to 0.883. This is data leakage: the higher score is not real. Our group-aware split removes the effect (0.828). Duplicates also give the duplicated documents more weight in the class priors and likelihoods.
- **Clustering:** 19 of 20 copies landed in the same cluster as their source. The duplicated documents form heavy groups that pull centroids towards them, and purity dropped from 0.606 to 0.549. Removing the detected copies restores the result.
- **Recommendation:** without filtering, 40.0% of the top-5 slots of the affected documents were taken by their own copies (two of five slots), which is useless to an analyst. The duplicates also artificially raise the precision@5 metric. Hiding results with cosine ≥ 0.9 removed all of them.
- **Detection quality:** all injected pairs were found with no false positives. The lowest Jaccard of an edited copy was 0.502, just above the 0.5 threshold: deleting 10% of words already breaks many 5-word shingles. That is why we also use the TF-IDF cosine. A heavily edited copy could be missed by Jaccard but is still caught by cosine.

Question 3: Explain the recommendation approach. Compare content-based and collaborative recommendation.

Our application uses **content-based** recommendation. Each document is a normalised TF-IDF vector, and the Top-K documents with the highest cosine similarity to the selected document (or to a typed query) are recommended, excluding the document itself and its near-duplicates. The shared terms explain every recommendation. On our corpus, 72.1% of the top-5 recommendations share the category of the source document.

	Content-based (used)	Collaborative filtering
Idea	Recommend items whose content is similar to the item the user is looking at	Recommend items that similar users liked / used (user-user or item-item co-occurrence)
Data needed	Only the item content (text) and its features	A user-item interaction matrix: views, clicks, ratings, bookmarks, ideally many per user and item
New items	Works immediately (no cold start for items)	Cold start: a new item has no interactions
Strengths	Explainable (shared terms), private, works for small user bases	Finds related items with different vocabulary; captures quality and popularity; more serendipity
Weaknesses	Only lexical similarity; ambiguous words; 'more of the same'; long general documents match many others	Needs a lot of interaction data; popularity bias; cold start for new users
Preferable when	Content is rich and interaction data is missing or sparse (our case)	There are many active users and logged interactions, e.g. a large news or e-commerce site

Table 17: Content-based vs collaborative recommendation

Our dataset contains no user interaction data, so collaborative filtering would need invented data. Content-based filtering is therefore the honest and appropriate choice. Its limitations are visible in our results: the ambiguity of 'agent' (Section 13.3), weak matches on generic words (*Data center* for *Stock market*), and no notion of which articles consultants actually found useful. Once the knowledge library logs which articles are opened together, a **hybrid** recommender could combine both: content scores for new articles, blended with collaborative scores as usage data grows.

Question 4: How do text mining, classification, clustering and recommendation together help domain experts make better decisions?

- **Text mining and corpus analytics** show what the library contains: the size of each practice area, document lengths, the most important terms, and data quality problems (reference noise, a stub article, off-domain documents, duplicates). Knowledge managers can decide what to clean and where content is missing. For example, Computing & Infrastructure has only 9 articles.
- **Classification** can tag new articles automatically. With about 83.1% cross-validated accuracy it is good enough to *suggest* a practice area, which a librarian then confirms. The posterior probabilities shown in the application identify uncertain cases for manual review. This saves manual effort and keeps tagging consistent.
- **Clustering** shows how the content actually groups, independent of our labels. Our clusters show that sequence-model and language articles overlap strongly with ML, and that web, services and security form a theme across categories. This can lead to a new practice area (e.g. 'Digital Platforms & Security') or to cross-references between areas.
- **Recommendation** helps consultants prepare faster. Starting from one relevant article (or a client question typed as free text) they get related reading with an explanation, without knowing the category structure.

- Together, these tools turn an unorganised folder of documents into a searchable, organised and monitored knowledge base. Experts spend less time searching and tagging and more time on analysis, and the evaluation metrics show how far each automatic result can be trusted.

Question 5: Compare Naive Bayes and K-Means. Key findings, limitations and improvements.

Results. Naive Bayes reached accuracy 0.857, macro precision 0.906, macro recall 0.827 and macro F1 0.844 on the test split, and 0.831 accuracy / 0.777 macro F1 in 5-fold cross-validation. K-Means with K = 5 reached purity 0.606 and Rand Index 0.757 (5-seed mean 0.648 / 0.774). Supervision makes a large difference. Naive Bayes recovers the human categories much more reliably than clustering. The Rand Index looks higher than purity mainly because most document pairs belong to different classes and are correctly separated (TN), so RI should not be read as '76% correct'.

Key findings.

- Both methods struggle with the same borderline documents: language-technology articles built on neural networks, and the heterogeneous Finance and Computing classes. These errors reflect real topic overlap and debatable labels rather than bugs.
- Noise cleaning and L2 length normalisation are essential for K-Means. Naive Bayes is much less sensitive to preprocessing.
- Duplicates can inflate classification accuracy through leakage (+5 percentage points in our experiment) and spoil recommendations. Group-aware splitting and duplicate filtering prevent this.
- K-Means results depend on initialisation and K. Lowest SSE was not the best match to the labels.

Limitations.

- The corpus is small (71 labelled documents, 21 in the test split), so one document changes accuracy by about 4.8 points, and the classes are imbalanced (22 vs 9 documents).
- Labels were assigned by us and some are debatable. There is only one label per document, although many articles span two areas.
- Wikipedia articles are long, encyclopaedic texts. Real consulting documents (reports, emails) are shorter and noisier in other ways.
- Bag-of-words TF-IDF ignores word order and meaning (synonyms, ambiguity such as 'agent').
- K-Means assumes roughly spherical, similarly sized clusters and needs K in advance. PCA shows only 7% of the variance in 2D, so the plot overlaps more than the real clusters do.
- The recommendation evaluation (category precision@K) is only a proxy. We have no user judgements.

Suggested improvements for real-world deployment.

- Collect more documents and labels, allow multi-label tagging, and use active learning: the librarian confirms the predictions with the lowest posterior probability first.
- Use feature selection (chi-square or mutual information) or Complement Naive Bayes for the imbalanced classes. Compare with a linear SVM or logistic regression.
- For clustering, choose K with the silhouette score, use spherical (cosine) K-Means or LSA/embeddings to reduce sparsity, and label clusters automatically with their top terms.
- Use sentence embeddings or LSA to capture synonyms in recommendations, add MMR re-ranking for diversity, and log user clicks to move towards a hybrid recommender.

- Run duplicate detection (MinHash/LSH for scale) on every new upload before indexing. Store documents and the index in a database with incremental index updates instead of recomputing in memory. Monitor accuracy over time as new topics appear, and re-train regularly.

17. Limitations

Besides the limitations discussed in Question 5: the application keeps the corpus and all models in memory and recomputes them when settings change, which is fine for about a hundred documents but would need incremental indexing for thousands. Uploaded documents are kept only for the browser session. Lemmatization uses WordNet without part-of-speech tagging (nouns, then verbs), so some forms are not reduced. The dataset is about 3.9 MB of text, below the optional 5 MB guideline, although the document count requirement is met.

18. Future Improvements

- Persistent storage of uploads and labels with an annotation screen for librarians.
- Multi-label classification and hierarchical categories.
- Semantic features (word or sentence embeddings) next to TF-IDF, and a hybrid recommender once usage data exists.
- Automatic cluster naming and trend monitoring over time (new topics appearing in the library).
- Scalable near-duplicate detection with MinHash and locality-sensitive hashing.

19. Conclusion

We built a complete, working Streamlit text mining system for a technology knowledge library. The system supports ingestion with duplicate detection, configurable preprocessing, manually implemented TF-IDF with an inverted index and cosine similarity, Naive Bayes classification (accuracy 0.857, CV 0.831), K-Means clustering (purity 0.606, Rand Index 0.757) and content-based recommendation (category precision@5 0.721). The experiments show that noise cleaning and length normalisation matter more than the choice between stemming and lemmatization, that supervised classification matches the human categories much better than clustering, and that duplicates must be handled to avoid misleading results. The whole workflow can be run and explored from the Streamlit front end.

20. References

C. D. Manning, P. Raghavan and H. Schuetze, *Introduction to Information Retrieval*, Cambridge University Press, 2008 (Ch. 6 TF-IDF and vector space model, Ch. 13 Naive Bayes, Ch. 16 flat clustering, Purity and Rand Index).

M. F. Porter, 'An algorithm for suffix stripping', *Program*, 14(3), pp. 130-137, 1980.

D. Arthur and S. Vassilvitskii, 'k-means++: The advantages of careful seeding', *Proc. ACM-SIAM SODA*, 2007.

A. Z. Broder, 'On the resemblance and containment of documents', *Proc. Compression and Complexity of Sequences*, 1997.

W. M. Rand, 'Objective criteria for the evaluation of clustering methods', *Journal of the American Statistical Association*, 66(336), pp. 846-850, 1971.

S. Bird, E. Klein and E. Loper, *Natural Language Processing with Python* (NLTK), O'Reilly, 2009.

F. Ricci, L. Rokach and B. Shapira (eds.), *Recommender Systems Handbook*, Springer, 2015.

Wikipedia contributors, English Wikipedia articles listed in data/processed/corpus_metadata.csv (exact revision URLs included), licensed under CC BY-SA 4.0.

Streamlit documentation, https://docs.streamlit.io

Appendix A - BITS Virtual Lab execution evidence

To be completed by the group: Virtual Lab screenshots are not yet included. Put the screenshots in `evidence/virtual_lab/` and run `python scripts/build_report.py` again. They will be inserted here automatically.

The screen recording is submitted as a separate file in the `evidence/` folder of the ZIP. The screenshots in the main part of this report were taken from a local run of the same code and are labelled 'local run'.

Appendix B - How to run

```
python -m venv .venv
.venv\Scripts\activate (Linux: source .venv/bin/activate)
pip install -r requirements.txt
streamlit run app.py

python -m pytest -q # 40 unit tests
python scripts/run_experiments.py # results + figures
python scripts/build_report.py # this PDF
```